



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Sede Bogotá
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial

Ingeniería de Software I

Entrega 5 - Módulo de Testing

Docente:

Oscar Eduardo Alvarez Rodriguez

Alumnos:

Sergio Andrés Ramírez Céspedes

Juan Eduardo Garzon Rodriguez

Julian Andres Melo Bustos

Diego Phelipe Morales Aldana

Índice

1. Introducción	3
2. Archivos de Testeo Ejecutados	3
2.1. apps/accounts/tests.py	3
2.2. apps/ingredients/tests.py	3
2.3. apps/recipes/tests.py	3
2.4. apps/accounts/tests.py	4
3. Resultados de la Ejecución	4
3.1. Tests Exitosos	4
3.1.1. apps/accounts/tests.py - (7 tests - 100 % exitosos)	4
3.1.2. apps/ingredients/tests.py - (16 tests - 11 exitosos)	5
3.1.3. apps/recipes/tests.py - (21 tests - 100 % exitosos)	5
3.1.4. apps/plans/tests.py - (14 tests - 10 exitosos)	5
3.2. Tests con Errores	5
3.2.1. apps/ingredients/tests.py	5
3.2.2. apps/plans/tests.py	6
4. Análisis de Impacto	6
4.1. Errores Comunes Identificados	7
4.2. Correcciones a Futuro	7

1. Introducción

El presente documento constituye el reporte de ejecución de pruebas de Pax-Saporis, correspondiente a la suite de pruebas unitarias y de integración implementada en Django (disponible en GitHub.)

Se ejecutaron las pruebas correspondientes a las funciones y casos de uso aplicables a las aplicaciones `accounts`, `ingredients`, `recipes` y `plans`. Se corrió el comando `python manage.py test --verbosity 2`, ejecutando un total de 53 tests distribuidos en cuatro archivos principales (`'apps/accounts/tests.py'`, `'apps/ingredients/tests.py'`, `'apps/recipes/tests.py'` y `'apps/plans/tests.py'`), en este caso no se separaron por miembro del proyecto de manera explícita, en su lugar, se tuvieron en cuenta los aportes de cada miembro para integrar los test con ayuda de herramientas de Inteligencia Artificial; según las funcionalidades contempladas en los Requerimientos Priorizados y la estructura del Backend.

2. Archivos de Testeo Ejecutados

Archivo	Aplicación	Descripción de Test	N° Tests
<code>apps/accounts/tests.py</code>	Cuentas	Tests de autenticación de usuarios en Django	7
<code>apps/ingredients/tests.py</code>	Ingredientes	Tests de endpoints y lógica de negocio	16
<code>apps/recipes/tests.py</code>	Recetas	Tests de vistas y validaciones HTTP	22
<code>apps/plans/tests.py</code>	Planes	Tests de planes y lógica de negocio	8

Tabla 1: Descripción de los Test Unitarios

2.1. `apps/accounts/tests.py`

Los tests planteados cubren las funcionalidades principales de autenticación, Registro de usuario, Login (éxito e inválido), Obtención del usuario actual (autenticado y no autenticado)m Logout.

2.2. `apps/ingredients/tests.py`

Los tests planteados son unitarios que abarcan vistas y modelos, enfocados en endpoints y lógica de negocio de ingredientes/recetas (validaciones, protección de defaults y cálculos automáticos de macros).

2.3. `apps/recipes/tests.py`

Los tests planteados son unitarios acerca de las vistas (ViewSets), usando mocks y SimpleTestCase, enfocados en validaciones HTTP, flujos exitosos y casos de error para endpoints de ingredientes y recetas.

2.4. apps/accounts/tests.py

Los tests planteados son unitarios que validan el cálculo de macros nutricionales en modelos PlanMeal y DailyPlan de Django.

3. Resultados de la Ejecución

Se ejecutaron un total de 53 tests en el entorno de Backend de Pax-Saporis:

- Tests exitosos (OK): 44
- Tests con ERROR: 9
- Tests fallidos (FAIL): 0
- Tests saltados (SKIPPED): 0

Con un porcentaje de éxito de aproximadamente 83 % (44/53). La ejecución se realizó sin problemas de configuración del entorno (la base de datos de prueba en memoria se creó correctamente y se aplicaron todas las migraciones). Sin embargo, se detectaron errores consistentes en las pruebas relacionadas con el cálculo de macronutrientes, específicamente en las clases que utilizan mocking de modelos Django. Los errores se concentran en dos archivos:

- apps/ingredients/tests.py (5 errores)
- apps/plans/tests.py (4 errores)

Estos fallos están relacionados principalmente con el uso incorrecto de MagicMock al asignar valores a campos ForeignKey de los modelos RecipeIngredient e PlanMeal.

3.1. Tests Exitosos

3.1.1. apps/accounts/tests.py - (7 tests - 100 % exitosos)

Todos los tests de autenticación (**AuthenticationTests**) pasaron correctamente:

- Registro de usuario
- Registro con contraseña no coincidente
- Login exitoso e inválido
- Obtención de usuario actual (autenticado y no autenticado)

- Logout

3.1.2. apps/ingredients/tests.py - (16 tests - 11 exitosos)

- TestAddIngredientValidationView (4 tests OK)
- TestAddIngredientSuccessView (3 tests OK)
- TestDefaultIngredientProtectionView (4 tests OK)
- TestRecipeIngredientMacroCalculation (0/5 tests OK)

3.1.3. apps/recipes/tests.py - (21 tests - 100 % exitosos)

- IngredientCreateViewUnitTests (5 tests OK)
- RecipeCreateViewUnitTests (5 tests OK)
- RecipeDetailViewUnitTests (5 tests OK)
- RecipeRemoveIngredientViewUnitTests (6 tests OK)

Todos los tests de vistas REST (creación, recuperación, eliminación de ingredientes y recetas, manejo de content-type, protección de registros default, etc.) pasaron correctamente.

3.1.4. apps/plans/tests.py - (14 tests - 10 exitosos)

- TestDailyPlanTotals (2 tests OK)
- TestDailyPlanMixedMeals (2 tests OK)
- TestPlanMealCalories (1/2 tests OK)
- TestPlanMealWithoutRecipe (0/2 tests OK)

Los tests relacionados con DailyPlan (sumatoria de macros) funcionan correctamente.

3.2. Tests con Errores

3.2.1. apps/ingredients/tests.py

Los 5 tests de TestRecipeIngredientMacroCalculation:

- test_all_macros_calculated_correctly

- `test_calories_calculated_correctly`
- `test_decimal_precision`
- `test_large_quantity_scales_correctly`
- `test_zero_quantity_returns_zero_macros`

Fallaron con el mismo tipo de error:

```
1 ValueError: Cannot assign "<MagicMock ...>" : "RecipeIngredient.ingredient" must be a "
  Ingredient" instance.
```

El helper `make_recipe_ingredient` intenta asignar un `MagicMock` directamente al campo `ForeignKey ingredient`, lo cual Django rechaza.

3.2.2. apps/plans/tests.py

Los siguientes test de `TestPlanMealCalories`:

- `test_plan_meal_calories_with_decimal_values`
- `test_plan_meal_calories_with_recipe`

Fallaron con el error:

```
1 ValueError: Cannot assign "<MagicMock ...>" : "PlanMeal.recipe" must be a "Recipe"
  instance.
```

Los siguientes test de `TestPlanMealWithoutRecipe`:

- `test_plan_meal_all_macros_without_recipe`
- `test_plan_meal_calories_without_recipe`

Fallaron adicionalmente con el error:

```
1 AttributeError: 'PlanMeal' object has no attribute '_state'
```

al intentar asignar `meal.recipe = None`.

4. Análisis de Impacto

Los 9 tests con errores impiden validar completamente funcionalidades críticas del sistema:

- **RF05** (Cálculo automático de calorías y macronutrientes de recetas), Afectado por los tests de `RecipeIngredient`.
- **RF12** (Visualización de totales en planes diarios), Afectado por los tests de `PlanMeal` y `DailyPlan`.
- **RF04** y **RF11** (Agregar ingredientes a recetas y asignar recetas a comidas), Indirectamente impactados, ya que dependen de cálculos correctos.

Aunque el 83 % de los tests (especialmente con vistas en recetas y ingredients, y la autenticación) funcionan correctamente, la capa de modelos relacionada con lógica de negocio presenta carencias en los tests. No se detectaron fallos en lógica de negocio real (solo problemas de testing), lo que indica que el código de producción probablemente funciona, pero la suite de pruebas generada con apoyo de Inteligencia Artificial no lo valida adecuadamente.

4.1. Errores Comunes Identificados

- Asignación directa de `MagicMock()` a campos `ForeignKey` (ingredient y recipe), violando las validaciones de tipo de Django.
- Uso de `__new__()` sin inicializar correctamente el estado interno del modelo (`_state`), causando `AttributeError` al asignar `None`.
- Tests excesivamente acoplados a la implementación interna de mocking en lugar de probar el comportamiento real de las propiedades calculadas.
- Falta de uso de herramientas especializadas para pruebas de modelos Django (como `model_bakery` o instancias reales en `TestCase`).

4.2. Correcciones a Futuro

Se busca con la implementación de las siguientes correcciones y cambios acelerar el avance en los requerimientos MUST relacionados con nutrición y planes de alimentación para después dar paso al desarrollo en firme del Frontend.

- Reemplazar los helpers de mocking por `model_bakery.baker.make()` o instancias reales de modelos en `TestCase`.
- Para casos de aislamiento puro, usar `Mock(spec=Ingredient)` y asignarlo correctamente.
- Preferir `django.test.TestCase` sobre `SimpleTestCase` cuando se prueban modelos con relaciones.

- Implementar un servicio o método dedicado en los modelos (`SimpleTestCase`) para centralizar la lógica de cálculos y facilitar su testeo.
- Usar `assertAlmostEqual(..., places=2)` consistentemente para valores decimales de macronutrientes.